ALG-18-19-RESUELTO.pdf

JUMIACOR

Algorítmica

2º Grado en Ingeniería Informática

Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada



Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.





Universidad de Granada
Departamento de Ciencias de la Computación e Inteligencia Artificial
E.T.S.I. Informática y de Telecomunicación, C/Periodista Daniel Saucedo Aranda s/n - 18071 - Granada (España)

Grado en Informática

Algorítmica

Curso 2018/2019. Convocatoria ordinaria de junio
6 de junio de 2019

1. (2 puntos) Calcular el orden de eficiencia en notación $O(\cdot)$ del algoritmo cuya expresión de tiempo es:

$$T(n) = 3T(\frac{n}{2}) + n\log_2(n)$$

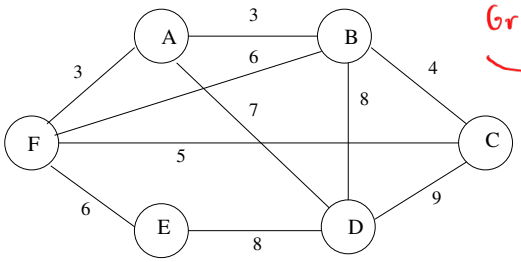
con $T(1) = 1$.

2. (2 puntos) Una estación de ITV consta de m líneas de inspección de vehículos iguales. Hay un total de n vehículos que necesitan inspección. En función de sus características, cada vehículo tardará en ser inspeccionado un tiempo t_i, $i = 1, \ldots, n$. Se desea encontrar la manera de atender a los n vehículos y acabar en el menor tiempo posible. Diseñad un algoritmo de exploración de grafos que determine cómo asignar los vehículos a las líneas. Especificad claramente, además del algoritmo, la representación del problema, la representación de la solución, las restricciones explícitas e implícitas, así como las posibles cotas a utilizar. B&B y Backtracking

3. (2 puntos) La empresa X ha decidido instalar una red de fibra óptica para interconectar sus seis centros de trabajo. El coste (positivo) del tendido de cable entre ellos aparece en la figura siguiente. Especificad un algoritmo que sea capaz de encontrar la forma de interconectar todos los centros con un coste mínimo, y aplicadlo para resolver el problema de la figura, describiendo el proceso paso a paso.

Greedy

Grafo Completo - PRIM
Grafo No Completo - KRUSKAL
Pero se puede usar cualquiera



4. (2 puntos) Tenemos que pagar la cuenta en un restaurante por valor de M euros, y disponemos de una cantidad ilimitada de monedas de n tipos diferentes, siendo $c[i]$ el valor de cada moneda de tipo i, $i = 1, \ldots, n$. Tenemos prisa y no queremos esperar que, en su caso, nos den la vuelta, pero como no hemos quedado muy satisfechos queremos pagar o la cantidad exacta M o la mínima cantidad posible mayor que M (o sea dejar ninguna propina o la mínima propina posible de acuerdo a las monedas que tenemos). Diseñar un algoritmo eficiente que determine la cantidad mínima que tenemos que pagar y cómo hacerlo.

Aplicadlo para resolver el siguiente caso: hay $n = 3$ tipos de monedas de valores 5, 7 y 13, y queremos pagar una cantidad de $M = 11$ unidades.

Cliente tacaño !!! Pr Dinámica minimizando el cobro

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

1. (2 puntos) Calcular el orden de eficiencia en notación $O(\cdot)$ del algoritmo cuya expresión de tiempo es:

$$T(n) = 3T(\frac{n}{2}) + n\log_2(n)$$

con $T(1) = 1$.

$T(n) = 3T(n/2) + n \cdot \log_2(n)$ / $n = 2^k$ $k = \log_2 n$

$T(2^k) = 3T(2^{k-1}) + 2^k \cdot k$

$T(k) = 3T(k-1) + 2^k \cdot k$

$(x-3) = (x-2)^2$

$C_1 \cdot 3^k + C_2 \cdot k \cdot 2^k + C_3 \cdot 2^k$

2º cambio de variable

$C_1 \cdot n^{\log_2 3} + C_2\, n \cdot \log_2(n) + C_3 \cdot n$

$O(n^{\log_2(3)})$

Tener en cuenta:

$c_1 \cdot k^{\log_2 n} + C_2 \cdot 2^k = c_1 \cdot n^{\log_2 k} + C_2 \cdot n$ /

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins

msi



CLAW A1M

Para jugársela tanto como tú estudiando el día de antes.



La MSI Claw A1M redefine el gaming portátil con su procesador Intel Core Ultra y tecnología XeSS para un rendimiento AAA, junto con diseño ergonómico, refrigeración avanzada, iluminación RGB personalizable, y la capacidad de conectar a una pantalla de sobremesa a través de Thunderbolt 4.

COMPRAR AHORA

Algorítmica



Banco de apuntes de la

WUOLAH

Comparte estos flyers en tu clase y consigue más dinero y recompensas

1. Imprime esta hoja
2. Recorta por la mitad
3. Coloca en un lugar visible para que tus compis puedan escanar y acceder a apuntes
4. Llévate dinero por cada descarga de los documentos descargados a través de tu QR

1. (2 puntos) Calcular el orden de eficiencia en notación $O(\cdot)$ del algoritmo cuya expresión de tiempo es:

$$T(n) = 3T(\frac{n}{2}) + n\log_2(n)$$

OTRA VEZ

con $T(1) = 1$.

$n = 2^k \quad k = \log_2(n)$

$T(2^k) = 3T(2^{k-1}) + 2^k \cdot k$

$(x-3) \quad (x-2)^2$

$C_1 \cdot 3^k + ((C_2 k + C_3) \cdot 2^k$

$C_1 \cdot n^{\log_2(3)} + C_2 \cdot n \cdot \log_2(n) + C_3 \cdot n \Rightarrow O(n^{\log_2(3)})$

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins

2. (2 puntos) Una estación de ITV consta de m líneas de inspección de vehículos iguales. Hay un total de n vehículos que necesitan inspección. En función de sus características, cada vehículo tardará en ser inspeccionado un tiempo t_i, $i = 1, \ldots, n$. Se desea encontrar la manera de atender a los n vehículos y acabar en el menor tiempo posible. Diseñad un algoritmo de exploración de grafos que determine cómo asignar los vehículos a las líneas. Especificad claramente, además del algoritmo, la representación del problema, la representación de la solución, las restricciones explícitas e implícitas, así como las posibles cotas a utilizar.

Exploración Grafos

No tiene por qué ser eficiente

- REPRESENTACIÓN DEL PROBLEMA Y SOLUCIÓN

"m" líneas de inspección, "n" vehículos a inspeccionar y "t" tiempo de cada coche.

* La posición de los vectores representan el coche, y la componente la info.
* La solución "S" tendría de componente la línea a la que irá el coche.

- RESTRICCIONES EXPLÍCITAS

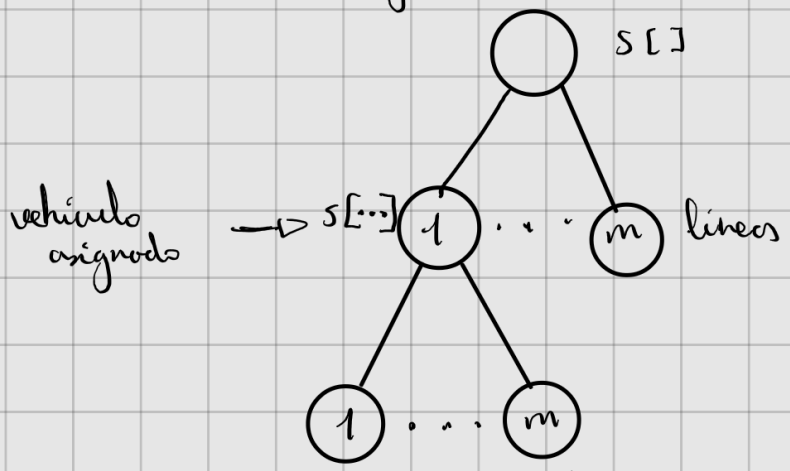
Las componentes del vector solución son la línea que se le asigna al coche, por lo tanto $1 \leq s[i] \leq m$

- RESTRICCIONES IMPLÍCITAS

Todas las componentes del vector solución "s" de tamaño "n" han de tener el valor de una línea (todos los coches asignados)

- ÁRBOL IMPLÍCITO

El factor de ramificación son las líneas que se le puede asignar a un vehículo (Profundidad).



- COTAS (PROBLEMA MINIMIZACIÓN)

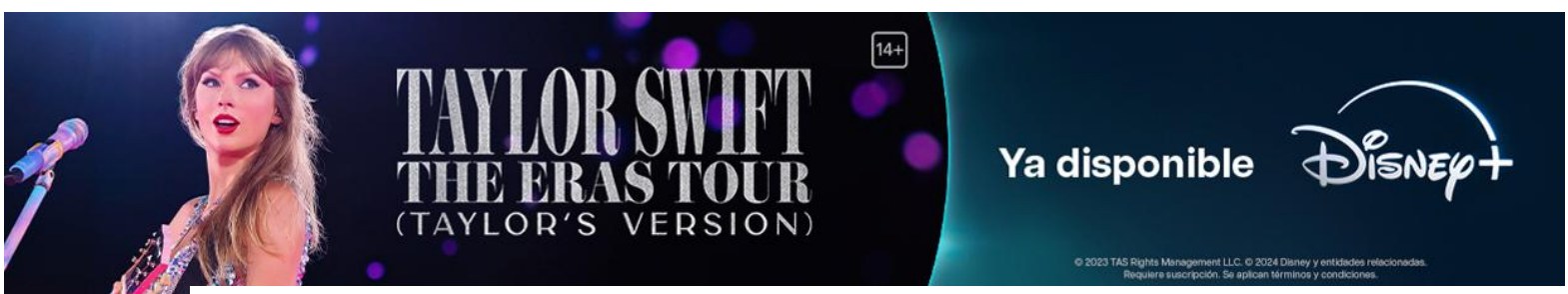
Cota global - mejor tiempo hasta el momento de una solución

Cota local (superior) - Dividimos la suma de lo que tardan los coches candidatos sin escoger entre las líneas que hay y sumamos el tiempo que llevamos

WUOLAH

Las descargas sin publicidad se realizan con las coins

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.



ALGORITMO MEJOR

```
void backITV(vector& m, vector& n, vector &t, vector &s, int asignados, float ti){
  float mejor=0; int n_coches; if (mejor==0) n_coches=n.size();
  if (asignados = n_coches) { // todos asignados
    if(mejor==0 || ti < mejor) mejor = ti ; //Cota global
    print "Solución encontrada" mejor, s;
  }
  else {
    for(int i=0; i< n_coches; i++){
      if (s[i]==0) //El coche no ha sido ya asignado (Factibilidad)
        s[i] = asignarLineaCoche(s, n, t[i], ti , m); //Calcula cuál línea
                                                      //es mejor
        // tiempo fue actualizado en la función
        n.erase(i);
        asignados ++;
        if (cotaLocal(s, t, n, ti) < mejor && mejor!=0)
          backITV(m, n, l, s, asignados, ti); //Poda región cota local
        }
        n.push_back(i); } Rollback
        asignados --;
      }
    }
  }
}
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Algoritmo

```
void backITV ( vector &c, vector &t, vector &s, int asignados, float tiempo ){
  float mejor = 0;        int total_coches = c.size();  // Sólo lo haremos la 1ª iteración
  if ( asignados == total_coches ) { // Se llega a una solución
    * if (mejor == 0) mejor = tiempo;
    * else if ( tiempo < mejor) mejor = tiempo; // Cota global
      print "Solución encontrada", tiempo, s;
      else {
          for ( int i = 0; i < total_coches ; i++ ) {
              if ( s[i] == 0 ) // Función Factibilidad
                  s[i] = asignar línea coche (s, tiempo, t[i] );
                  tiempo += t[i];
                  c.erase (c[i]);
                  asignados ++;
                  if ( tiempo < mejor && mejor != 0){ // Poda según cota
                      backITV (c, t, s, asignados, tiempo);
                  }
                  s[i] = 0;
                  tiempo -= t[i];
                  c.push_back (c[i]);
                  asignados --;
              }
          }
      }
  }
}
```

* De la línea que minimiza el tiempo global

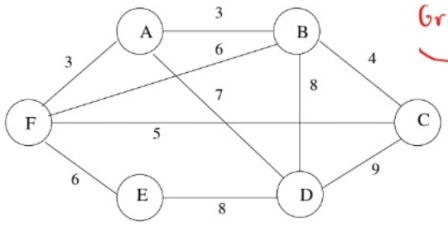
Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins

3. (2 puntos) La empresa X ha decidido instalar una red de fibra óptica para interconectar sus seis centros de trabajo. El coste (positivo) del tendido de cable entre ellos aparece en la figura siguiente. Especificad un algoritmo que sea capaz de encontrar la forma de interconectar todos los centros con un coste mínimo, y aplicadlo para resolver el problema de la figura, describiendo el proceso paso a paso.

Greedy



Grafo Completo - PRIM
Grafo No Completo - KRUSKAL
Pero se puede usar cualquiera

Problema Árbol Generador Mínimal
Grafo ponderado ✓
Grafo no dirigido ✓
Grafo conexo ✓
* Utilizaré Kruskal

- CANDIDATOS: Líneas de fibra (Aristas)
- FUNCIÓN SOLUCIÓN: Cuando el conjunto de líneas conecta todos los centros (V) y además el número de líneas seleccionadas es igual al número de centros menos uno.
- FUNCIÓN OBJETIVO: Suma de costes mínima.
- FUNCIÓN FACTIBILIDAD: No se formen ciclos.
- FUNCIÓN SELECCIÓN: Línea (arista) de menor coste.

```
set<arco> KruskalGreedy (Grafo[V,A]) { // Grafo con vértices y aristas
    set<arcos> C(A); // CANDIDATOS
    set<arcos> S; // SOLUCIÓN VACÍA
    C.sort(); // ORDENAMOS LOS CANDIDATOS
    while (!C.empty() && S.size() == V.size()-1) {
        x = C.first(); // SELECCIONO EL DE MENOR COSTO
        C.erase(x); // SE QUITA DE LOS CANDIDATOS
        if (!HayCiclos(S, x)) // FUNCIÓN FACTIBILIDAD
            S.insert(x); // INSERTAMOS
    }
    if (S.size() == V.size()-1) return S; // Devolvemos solución
    else
        return "No solución"; // No solución
```

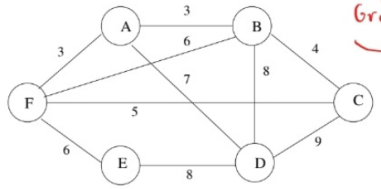
Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins

3. (2 puntos) La empresa X ha decidido instalar una red de fibra óptica para interconectar sus seis centros de trabajo. El coste (positivo) del tendido de cable entre ellos aparece en la figura siguiente. Especificad un algoritmo que sea capaz de encontrar la forma de interconectar todos los centros con un coste mínimo, y aplicadlo para resolver el problema de la figura, describiendo el proceso paso a paso.

Greedy



Grafo Completo - PRIM
Grafo No Completo - KRUSKAL
Pero se puede usar cualquiera

CANDIDATOS - Aristas aún no escogidas en la solución (fibra)

SELECCIONADOS - Aristas seleccionadas para la solución

F. FACTIBILIDAD - Que no se formen ciclos

F. SELECCIÓN - La arista de menor coste de los candidatos

F. OBJETIVO - "Árbol Generador Mínimo"

F. SOLUCIÓN - Cuando el número de aristas seleccionadas sea igual al número de vértices del grafo menos uno.

```
set<arcos> KruskalFTTH ( Grafo (V, A)) {
    set<arcos> C (A) ; //CANDIDATOS
    set<arcos> S ;     // Seleccionados
    C. sort () ;       // Ordenamos de menor a mayor coste
    while (!C. empty() || S. size() == V.size - 1) {
        x = C. first
        C. delete (x);
        if (x. NoHayCiclos (S)) {
            S. push_back (x);
        }
    }
    if ( S. size() == V.size() - 1) {
        return S;
    } else {
        return {};
    }
}
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins



4. (2 puntos) Tenemos que pagar la cuenta en un restaurante por valor de M euros, y disponemos de una cantidad ilimitada de monedas de n tipos diferentes, siendo $c[i]$ el valor de cada moneda de tipo i, $i = 1, \ldots, n$. Tenemos prisa y no queremos esperar que, en su caso, nos den la vuelta, pero como no hemos quedado muy satisfechos queremos pagar o la cantidad exacta M o la mínima cantidad posible mayor que M (o sea dejar ninguna propina o la mínima propina posible de acuerdo a las monedas que tenemos). Diseñar un algoritmo eficiente que determine la cantidad mínima que tenemos que pagar y cómo hacerlo.

   Aplicadlo para resolver el siguiente caso: hay $n = 3$ tipos de monedas de valores 5, 7 y 13, y queremos pagar una cantidad de $M = 11$ unidades.

PR. Dinámica Minimizando sobre nº monedas

Casos base características

1) REPRESENTACIÓN DEL PROBLEMA

- Secuencia de decisiones: se utiliza o no una moneda

M(k, m) → k: tipo de moneda ; m: cantidad actual a llegar

2) DEMOSTRACIÓN DE LA OPTIMALIDAD DE BELMAN

1) Suponemos que $x_1 \ldots x_n$ (monedas seleccionadas) es una solución óptima para el problema M[k, M].

2) Suponemos por contradicción que $x_1 \ldots x_{n-1}$ no es óptima:
   - Si $x_n = 0$; M[k-1, M]
   - Si $x_n = 1$; M[k-1, M-m_n]

3) En el caso $x_n = 0$ asumo que existe una solución $y_1 \ldots y_{n-1}$ para M[k-1, M] con "sum_1^{n-1} y_i < sum_1^{n-1} x_i" y "sum_1^{n-1} $y_i \cdot c_i = M - x_n \cdot c_n$"

4) Pero entonces si $y_n = x_n$ tenemos que: "sum_1^{n} $y_i \cdot c_i = M$" luego $y_1 \ldots y_n$ es una solución para M[k, M] y además "sum_1^{n} y_i < sum_1^{n} x_i" luego no sería óptima.

3) DEFINICIÓN RECURSIVA

$$M[k, M] \begin{cases} M[k, m] = M[k-1, m] & \text{si } c_i > m \text{ (Valor de la moneda mayor que el monto)} \\ M[k, m] = \min(M[k, m-c_i] + c_i \,,\, M[k-1, m]) & \text{en otro caso} \end{cases}$$

nº ilimitado de monedas

minimizando valor, no el nº de monedas

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

CASOS BASE: * Mientras que en el problema de las monedas "original" si no llegabamos al valor "m" con la moneda, no dábamos solución, aquí sí lo daremos

La matriz tendrá dimensión M[k+1, M+1]:

Primera Columna: M[i,0] = 0

Primera Fila: M[0,j] = 0

Segunda Fila (1ª moneda): M[0,j] = M[0, max(j-C[0], 0)] + C[0]

↑ para no salirse de la matriz

Se cerá mejor en la tabla

4) PSEUDOCÓDIGO

```
for (int i=0; i<K; i++)
    M[i,0] = 0
for (int j=0; j<M; j++) {
    M[0,j] = 0;
    M[1,j] = M[0, max(j-C[0], 0)] + C[0];
}
for (int i=1; i<K; i++) {
    for (int j=2; j<M; j++) {
        M[i,j] = min(M[i,j-C[i]] + C[i], M[i-1,j]);
    }
}
```

} CASOS BASE

} Los demás

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins

5/6) RECONSTRUCCIÓN DE LA SOLUCIÓN / TABLA

if M[k, m] == M[k-1, m] // No se usa la moneda y se pasa a estudiar:

M[k-1, m]

else // Se usa la moneda y se pasa a estudiar:

M[k, m-ki]

[Tabla] monedas {5, 7, 13} M=11

m → \ k ↓	0	1	2	3	4	5	6	7	8	9	10	11
0	0	0	0	0	0	0	0	0	0	0	0	0
5(1)	0	5	5	5	5	5	10	10	10	10	10	15
7(2)	0	5	5	5	5	5	7	7	10	10	10	15
13(3)	0	5	5	5	5	5	7	7	10	10	10	15

{5, 5, 5}

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins

5. (2 puntos) Un aficionado al póker juega una partida cada día, y anota los euros que gana o pierde. Por ejemplo, la siguiente tabla muestra los resultados del último mes.

L	M	X	J	V	S	D
					29	-7
14	21	30	-47	1	7	-39
23	-20	-36	-41	27	- 34	7
48	35	-46	-16	32	18	5
-33	27	28	-22	1	-20	-42

Podemos ver que empezó el mes con una ganancia de 29 euros, y terminó con una pérdida de 42. El beneficio total obtenido en el mes (la suma de todas las ganancias y pérdidas) es -50 euros. Analizando la información, el jugador se podría dar cuenta de que si hubiera empezado a jugar el día 16 y terminado el día 26, habría maximizado sus ganancias, obteniendo 105 euros.

En general, dado un vector de ganancias/pérdidas de longitud n, se desea encontrar el subvector sobre el cual se consigue el beneficio total máximo. Diseñad un algoritmo que realice esta tarea de forma lo más eficiente posible.

Duración del examen: 2 horas y 30 minutos.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins



5. (2 puntos) Un aficionado al póker juega una partida cada día, y anota los euros que gana o pierde. Por ejemplo, la siguiente tabla muestra los resultados del último mes.

L	M	X	J	V	S	D
					29	-7
14	21	30	-47	1	7	-39
23	-20	-36	-41	27	- 34	7
48	35	-46	-16	32	18	5
-33	27	28	-22	1	-20	-42

Podemos ver que empezó el mes con una ganancia de 29 euros, y terminó con una pérdida de 42. El beneficio total obtenido en el mes (la suma de todas las ganancias y pérdidas) es -50 euros. Analizando la información, el jugador se podría dar cuenta de que si hubiera empezado a jugar el día 16 y terminado el día 26, habría maximizado sus ganancias, obteniendo 105 euros.

En general, dado un vector de ganancias/pérdidas de longitud n, se desea encontrar el subvector sobre el cual se consigue el beneficio total máximo. Diseñad un algoritmo que realice esta tarea de forma lo más eficiente posible.

D y V
rollo búsqueda binaria

```
int max (int x, int y) {
    return (x > y) ? x : y;
}

int maximoBeneficio (int v, int low, int high)
{
    if (high <= low) {
        return v[low]; // Se termina (0 o 1 componente)
    }
*   int mitad = (low + high) / 2;
    int left_max = -∞ ; // Buscamos la suma mayor del subarreglo
    int sum = 0;        // izquierdo
    for (int i = mitad; i >= low; i--)
        sum += v[i];
        if (sum > left_max) {
            left_max = sum;
        }
    }
```

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
int right_max = INT_MIN; //Buscamos la suma mayor del
int sum = 0;             // subarreglo derecho
for (int i = mid + 1; i <= high; i++) {
    sum += v[i];
    if (sum > right_max) {
        right_max = sum;
    }
}
//Encuentro recursivamente la suma máxima del subarreglo para la
izquierda y derecha, y tomo el máximo.
int max_left_right = max(maximoBeneficio(v, low, mitad),
                         maximoBeneficio(v, mitad+1, high));

// Devuelvo el máximo de los "3" cálculos
return max(max_left_right, left_max, right_max)
}
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Las descargas sin publicidad se realizan con las coins